In [1]:
```
from pyspark.sql import SparkSession
import getpass
username = getpass.getuser()
spark = SparkSession. \
builder. \
config('spark.ui.port', '0'). \
config("spark.sql.warehouse.dir", f"/user/{username}/warehouse"). \
enableHiveSupport(). \
master('yarn'). \
getOrCreate()
```

In [2]:
```
spark
```

Out[2]:

SparkSession - hive

SparkContext

Spark UI

Version	`v3.1.2`
Master	`yarn`
AppName	`pyspark-shell`

1. Let's define some words in a list and find the frequency of each word

In [3]:
```
# local data in gateway node here
words = ("big","Data","Is","SUPER","Interesting","BIG","data","IS","A","Trendi
```

1. the above words are in local machine here . this is not parallel here
2. Since it is normal collection only , it is running on a single machine on gayeway node
3. We need to create a rdd out of it where rdd means a collection which is distributed across the cluster .

In [4]:
```
# so use the spark.sparkcontext and apply parallelize so this can become the r
# created an rdd here
# we can develop entire logic on this RDD and check whether it is working fine
# if it works fine , then instead of parallelize we can  start work on text fi
words_rdd = spark.sparkContext.parallelize(words)
```

In [5]:
```
# normalise all the words -- convert all to lower case letter
# apply the map here as we want to work on every word
words_normalized = words_rdd.map(lambda x:x.lower())
```

In [6]:
```
words_normalized.collect() # to print the above result
```

Out[6]:
```
['big',
 'data',
 'is',
 'super',
 'interesting',
 'big',
 'data',
 'is',
 'a',
 'trending',
 'technology']
```

In [7]:
```
mapped_words = words_normalized.map(lambda x:(x,1))
```

In [8]:
```
aggregated_result = mapped_words.reduceByKey(lambda x,y:x+y)
```

In [9]:
```
aggregated_result.collect()
```

Out[9]:
```
[('is', 2),
 ('trending', 1),
 ('technology', 1),
 ('super', 1),
 ('interesting', 1),
 ('big', 2),
 ('data', 2),
 ('a', 1)]
```

In [10]:
```
spark.sparkContext.parallelize(words).map(lambda x:x.lower()).map(lambda x:(x,
```

Out[10]:
```
[('is', 2),
 ('trending', 1),
 ('technology', 1),
 ('super', 1),
 ('interesting', 1),
 ('big', 2),
 ('data', 2),
 ('a', 1)]
```

In [ ]:
```

```

In [11]:
```
# here we can perform the chaining of the functions here
# spark.sparkContext.parallelize(words).map(lambda x:x.lower()).map(lambda x:(
# we can give space backslash for clarity  which means continuation
# no space after backslash

result = spark. \
sparkContext. \
parallelize(words). \
map(lambda x:x.lower()). \
map(lambda x:(x,1)). \
reduceByKey(lambda x,y:x+y)
```

In [12]:
```
result.collect()
```

Out[12]:
```
[('is', 2),
 ('super', 1),
 ('interesting', 1),
 ('trending', 1),
 ('technology', 1),
 ('big', 2),
 ('data', 2),
 ('a', 1)]
```

In [ ]:
```

```